

AIRPORT TRAFFIC PREDICTION

using Apache Spark (PySpark MLlib)

INT315 – CLUSTER COMPUTING

Project Report

Distributed data analysis and next-day forecasting of IFR movements at 331 European airports using Linear Regression, Random Forest and Gradient-Boosted Trees

School of AI and Emerging Technology

Student name	Rai Yashmit Ashok
Registration number	12313527
Section / Roll number	RK4M2310A 21
Faculty / Instructor	Amandeep Kaur
Submission date	27/10/26

Declaration:

I declare that this Assignment is my individual work. I have not copied it from any other students' work or from any other source except where due acknowledgement is made explicitly in the text, nor has any part been written for me by any other person.

Student's Signature: Yashmit

ACKNOWLEDGEMENT

We acknowledge the guidance provided by Ms. Amandeep Kaur for the INT315 Cluster Computing academic project. We also acknowledge the EUROCONTROL-based airport traffic dataset (airport_traffic_2025.csv) made available on Kaggle, which supplied the daily flight records used in this work. The report applies cluster computing concepts using Apache Spark (PySpark) and analyses the information across the full 2025 dataset.

EXECUTIVE SUMMARY

This report presents the "Airport Traffic Prediction" project, a distributed data analysis and forecasting pipeline built with Apache Spark (PySpark MLlib) in Google Colab. Its design combines Spark DataFrames, lazy evaluation and partitioned processing, lag-based feature engineering (previous-day and previous-week flights), a time-based train/test split, and a comparison of Linear Regression, Random Forest and Gradient-Boosted Trees against a simple previous-day baseline. The supplied dataset contains 116,020 daily records covering 331 airports in 42 countries across 365 days of 2025, with no duplicate rows. After cleaning and lag-feature creation, 110,260 records were used, of which 81,858 were used for training (before 1 October 2025) and 28,402 for testing (1 October to 31 December 2025). Linear Regression gave the best result, reducing the mean absolute error from 17.43 to 10.37 flights per airport-day and the RMSE from 33.75 to 21.17 compared with the previous-day baseline. The fitted model gives about 70% weight to the flights seven days earlier and about 30% to the previous day, showing that weekly seasonality is the main driver of airport traffic. The selected model was then used to forecast next-day flights (1 January 2026) for 279 airports, and the predictions were saved as CSV files.

Table of Contents

No.	Section	Page
1	Project Title	4
2	Problem Statement	4
3	Objective of the Project	4
4	Dataset Description	5
5	Dataset Source	6
6	Tools & Technologies Used	7
7	Implementation / Methodology	8
8	Complete Code	10
9	Execution Screenshots	14
10	Output / Result Screenshots	17
11	Result Analysis	22
12	Conclusion	23
13	References	24

Click a section name or page number to jump to that section.

1. Project Title

AIRPORT TRAFFIC PREDICTION

A distributed analysis and next-day forecasting pipeline for daily IFR flight movements at 331 European airports (2025 data).

2. Problem Statement

Airports, airlines and air-navigation service providers must know in advance how many flights will use each airport on a given day. The figure drives decisions on runway and gate allocation, ground-handling and security staffing, air-traffic-control capacity and slot management. Under-estimating demand leads to congestion and delay; over-estimating it wastes resources.

Operational data of this kind is large and keeps growing. The 2025 file alone has more than 116,000 airport-day records for 331 airports in 42 countries, and a full multi-year archive across all European airports would quickly outgrow a single-machine, in-memory workflow. The problem addressed by this project is therefore twofold: (a) **how to process and explore large airport-traffic data using a cluster computing framework**, and (b) **how to predict the number of flights at every airport for the next day** from the airport's recent traffic history.

3. Objective of the Project

- Load and explore the 2025 airport-traffic dataset with Apache Spark DataFrames (schema, statistics, missing values, duplicates, monthly totals, busiest airports).
- Clean the data and engineer two lag features per airport: the previous day's flights (*yesterday*) and the flights seven days earlier (*last_week*) to capture the weekly cycle.
- Train and compare three Spark MLlib regression models – Linear Regression, Random Forest and Gradient-Boosted Trees – against a simple previous-day baseline, using a time-based train/test split.
- Evaluate the models with MAE and RMSE and select the best one automatically.
- Use the selected model to forecast the next-day flights (1 January 2026) for every airport and save the predictions as CSV files.
- Demonstrate the cluster-computing concepts of distributed DataFrames, lazy evaluation, partitioned joins/aggregations and parallel model training.

4. Dataset Description

The dataset (`airport_traffic_2025.csv`) contains one record per airport per day and reports the number of Instrument Flight Rules (IFR) movements – departures, arrivals and their total. It covers the whole of 2025.

Property	Value
File / size	airport_traffic_2025.csv, about 7.4 MB
Rows x columns	116,020 x 13
Period	1 January 2025 – 31 December 2025 (365 distinct dates)
Airports	331 distinct ICAO airport codes
Countries / states	42
Duplicate rows	0
Missing values	0 in the first ten columns; 81,892 (70.6 %) in each of the three *_IFR_2* columns

Column	Type	Description
YEAR	integer	Reference year (2025)
MONTH_NUM / MONTH_MON	integer / string	Month as a number and as a 3-letter code
FLT_DATE	date	Date of the traffic count (YYYY-MM-DD)
APT_ICAO	string	4-letter ICAO airport designator (e.g. EGLL)
APT_NAME	string	Airport name (e.g. London - Heathrow)
STATE_NAME	string	Country in which the airport is located
FLT_DEP_1	integer	Number of IFR departures
FLT_ARR_1	integer	Number of IFR arrivals
FLT_TOT_1	integer	**Total IFR movements (departures + arrivals) – the prediction target**
FLT_DEP_IFR_2, FLT_ARR_IFR_2, FLT_TOT_IFR_2	integer	Airport-operator-reported IFR counts; available for only 29.4 % of rows, so not used for modelling

Key statistics of the target `FLT_TOT_1`: mean 148.15 movements per airport-day, standard deviation 250.12, median 39, maximum 1,688. The distribution is strongly right-skewed: many small regional

airports record a few dozen flights a day while hubs such as Istanbul, Amsterdam and London Heathrow exceed 1,300. In every row, FLT_TOT_1 equals FLT_DEP_1 + FLT_ARR_1.

5. Dataset Source

Dataset name: Airport Traffic Dataset (file `airport_traffic_2025.csv`), published on Kaggle.

Dataset link: https://www.kaggle.com/datasets/nilesh2042/airport-traffic-dataset?select=airport_traffic_2025.csv

Original data provider: The column layout of the file (IFR departures, arrivals and totals per airport per day, with ICAO airport codes) corresponds to the airport-traffic statistics of the EUROCONTROL Aviation Intelligence Unit, which publishes this type of data at https://www.ansperformance.eu/data/.

6. Tools & Technologies Used

Category	Tool / technology	Purpose
Cluster computing framework	Apache Spark (PySpark), local mode	Distributed in-memory processing of the DataFrame; lazy evaluation, DAG scheduling and partitioned execution
Programming language	Python 3	Driver program
Data processing	pyspark.sql (SparkSession, DataFrame, functions, types)	Reading CSV, schema inference, cleaning, aggregation, joins, lag features
Machine learning	pyspark.ml: VectorAssembler, LinearRegression, RandomForestRegressor, GBTRegressor, RegressionEvaluator	Feature vector assembly, model training, prediction and MAE/RMSE evaluation
Execution environment	Google Colab (Linux, Python notebook)	Runs the Spark session; files read from and written to /content
Data format	CSV (input and output)	airport_traffic_2025.csv in; test predictions and next-day forecast out
Reporting	matplotlib, pandas, ReportLab / docx	Charts and this report (not part of the Spark pipeline)

Cluster-computing perspective. Spark splits a DataFrame into partitions that executors process in parallel under the control of a driver. Transformations such as `select`, `filter` and `join` are lazy; they only build an execution plan (DAG) that runs when an action such as `count()`, `show()` or `write` is called. Wide operations (`groupBy`, `countDistinct`, `join`) shuffle data between partitions, and MLlib trains Random Forest and GBT models by distributing the tree-building work. In Colab the session runs in local mode (all cores of one machine act as executors); the same program can be submitted unchanged to a real Spark cluster (standalone, YARN or Kubernetes) simply by changing the master/deployment settings.

7. Implementation / Methodology

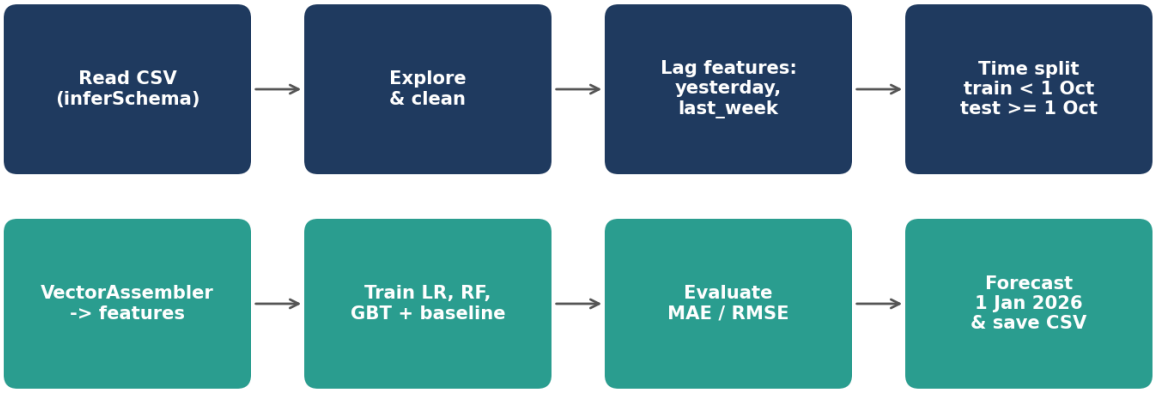


Figure 1. Overall pipeline of the project.

Step 1 – Start Spark and read the data. A `SparkSession` named AirportTrafficModels is created and the CSV is read with `header=True` and `inferSchema=True`, so Spark detects integer, date and string columns.

Step 2 – Exploratory data analysis. The program prints the row/column counts, schema, first rows and a statistical summary (`describe()`) of all numeric columns; counts missing values (null or empty string) per column; counts duplicate rows; counts distinct airports, dates and countries; aggregates monthly flight totals; and ranks the ten busiest airports with `groupBy`/`agg`.

Step 3 – Data cleaning. Only date, airport code, airport name and the target `FLT_TOT_1` (cast to double and renamed *flights*) are kept. Strings are trimmed, null rows and negative counts are removed, and duplicates on (date, airport) are dropped. 116,020 rows remain.

Step 4 – Feature engineering. Two lag tables are built by shifting each date forward with `date_add` by 1 day (*yesterday*) and 7 days (*last_week*) and are left-joined back on (airport, date). Rows that lack either value (the first week of the year per airport) are dropped, leaving 110,260 rows.

Step 5 – Time-based train/test split. To avoid leaking future information, rows before 1 October 2025 are used for training (81,858 rows) and rows from 1 October 2025 onwards for testing (28,402 rows).

Step 6 – Vector assembly. `VectorAssembler` combines *yesterday* and *last_week* into a single *features* vector as required by Spark ML.

Step 7 – Baseline and model training. A baseline predicts that today equals yesterday. Three regressors are then trained: Linear Regression (defaults), Random Forest (20 trees, max depth 5, seed 42) and Gradient-Boosted Trees (20 iterations, max depth 3, seed 42). Negative predictions are clipped to zero because flight counts cannot be negative.

Step 8 – Evaluation and model selection. `RegressionEvaluator` computes MAE and RMSE on the test set for the baseline and each model; the model with the lowest RMSE is kept as *best_model* and the results table is sorted by RMSE.

Step 9 – Next-day forecast. The last date in the data (31 December 2025) supplies *yesterday*; the value of 25 December 2025 supplies *last_week* for the forecast date 1 January 2026. The best model predicts the flights of every airport, and the 20 highest forecasts are displayed.

Step 10 – Saving the results. The test predictions and the next-day forecast are written as single-file CSVs (`coalesce(1)`) to /content/airport_results/test_predictions and /content/airport_results/next_day_predictions, and the Spark session is stopped in a `finally` block.

8. Complete Code

The complete PySpark program (`airport_flight_prediction.py`, 254 lines) is reproduced below exactly as executed.

```
# Airport flight prediction using three simple PySpark ML models
# Includes basic data analysis before training the models.

from datetime import timedelta
from pyspark.sql import SparkSession
from pyspark.sql import functions as F
from pyspark.sql.types import NumericType
from pyspark.ml.feature import VectorAssembler
from pyspark.ml.regression import LinearRegression, RandomForestRegressor, GBTRegressor
from pyspark.ml.evaluation import RegressionEvaluator

# Change these paths if your files are somewhere else
INPUT PATH = "/content/airport traffic 2025.csv"
OUTPUT PATH = "/content/airport results"

spark = SparkSession.builder.appName("AirportTrafficModels").getOrCreate()
spark.sparkContext.setLogLevel("ERROR")

try:
    # 1. Read the CSV
    raw = (
        spark.read
        .option("header", True)
        .option("inferSchema", True)
        .csv(INPUT PATH)
    )

    # ==================== DATA ANALYSIS ====================
    print("\n========== 1. BASIC DATA INFORMATION ==========")
    print("Number of rows:", raw.count())
    print("Number of columns:", len(raw.columns))
    print("Column names:", raw.columns)

    print("\nColumn data types:")
    raw.printSchema()

    print("\nFirst 10 rows:")
    raw.show(10, truncate=False)

    print("\n========== 2. STATISTICAL SUMMARY ==========")
    numeric cols = [
        field.name for field in raw.schema.fields
        if isinstance(field.dataType, NumericType)
    ]

    if numeric cols:
        raw.select(*numeric cols).describe().show(truncate=False)
    else:
        print("No numeric columns were detected.")

    print("\n========== 3. MISSING VALUES ==========")
    raw.select([
        F.sum(
            F.when(
                F.col(c).isNull() | (F.trim(F.col(c).cast("string")) == ""),
                1
            ).otherwise(0)
        ).alias(c)
        for c in raw.columns
    ]).show(truncate=False)

    print("\n========== 4. DUPLICATE ROWS ==========")
    print("Duplicate full rows:", raw.count() - raw.dropDuplicates().count())
```

```
64
65      print("\n========== 5. DATASET COUNTS ==========")
66      raw.agg(
67          F.countDistinct("APT ICAO").alias("unique airports"),
68          F.countDistinct("FLT DATE").alias("unique dates"),
69          F.countDistinct("STATE NAME").alias("unique states or countries")
70      ).show()
71
72      print("\n========== 6. MONTHLY FLIGHT TOTALS ==========")
73      raw.withColumn(
74          "month", F.month(F.to date(F.col("FLT DATE").cast("string")))
75      ).groupBy("month").agg(
76          F.sum(F.col("FLT TOT 1").cast("double")).alias("total flights")
77      ).orderBy("month").show(12)
78
79      print("\n========== 7. TEN BUSIEST AIRPORTS ==========")
80      raw.groupBy("APT ICAO", "APT NAME").agg(
81          F.sum(F.col("FLT TOT 1").cast("double")).alias("total flights"),
82          F.round(F.avg(F.col("FLT TOT 1").cast("double")), 2).alias("average flights")
83      ).orderBy(F.desc("total flights")).show(10, truncate=False)
84
85      # ==================== PREPARE DATA FOR ML ====================
86      print("\n========== 8. PREPARE DATA FOR MACHINE LEARNING ==========")
87      df = raw.select(
88          F.to date(F.col("FLT DATE").cast("string")).alias("date"),
89          F.trim(F.col("APT ICAO").cast("string")).alias("APT ICAO"),
90          F.trim(F.col("APT NAME").cast("string")).alias("APT NAME"),
91          F.col("FLT TOT 1").cast("double").alias("flights")
92      )
93
94      df = df.dropna().filter(F.col("flights") >= 0)
95      df = df.dropDuplicates(["date", "APT ICAO"])
96
97      print("Clean data sample:")
98      df.show(5, truncate=False)
99      print("Rows after cleaning:", df.count())
100
101     # 9. Add the flight count from yesterday and from 7 days ago
102     yesterday = df.select(
103         "APT ICAO",
104         F.date add("date", 1).alias("date"),
105         F.col("flights").alias("yesterday")
106     )
107
108     last week = df.select(
109         "APT ICAO",
110         F.date add("date", 7).alias("date"),
111         F.col("flights").alias("last week")
112     )
113
114     data = df.join(yesterday, ["APT ICAO", "date"], "left")
115     data = data.join(last week, ["APT ICAO", "date"], "left")
116     data = data.dropna(subset=["yesterday", "last week"])
117
118     print("Rows with both previous-day and previous-week data:", data.count())
119
120     # 10. Train on earlier dates and test on later dates
121     # This cutoff assumes the CSV contains data from 2025.
122     train = data.filter(F.col("date") < F.to date(F.lit("2025-10-01")))
123     test = data.filter(F.col("date") >= F.to date(F.lit("2025-10-01")))
124
125     if train.limit(1).count() == 0 or test.limit(1).count() == 0:
126         raise ValueError(
127             "Training or test data is empty. Check the dates and the 2025-10-01 cutoff."
128         )
129
130     print("Training rows:", train.count())
131     print("Testing rows:", test.count())
132
133     # 11. Put the two input columns into the format Spark ML needs
134     assembler = VectorAssembler(
135         inputCols=["yesterday", "last_week"],
```

```
        outputCol="features"
    )

    train = assembler.transform(train)
    test = assembler.transform(test)

    # 12. Compare a simple baseline with three machine-learning models
    # The baseline just assumes today's flights will equal yesterday's.
    mae evaluator = RegressionEvaluator(
        labelCol="flights", predictionCol="prediction", metricName="mae"
    )
    rmse evaluator = RegressionEvaluator(
        labelCol="flights", predictionCol="prediction", metricName="rmse"
    )

    baseline rmse = RegressionEvaluator(
        labelCol="flights", predictionCol="yesterday", metricName="rmse"
    ).evaluate(test)
    baseline mae = RegressionEvaluator(
        labelCol="flights", predictionCol="yesterday", metricName="mae"
    ).evaluate(test)

    results = [("Previous-day baseline", baseline mae, baseline rmse)]

    models = [
        ("Linear Regression", LinearRegression(
            featuresCol="features", labelCol="flights"
        )),
        ("Random Forest", RandomForestRegressor(
            featuresCol="features", labelCol="flights",
            numTrees=20, maxDepth=5, seed=42
        )),
        ("Gradient-Boosted Trees", GBTRegressor(
            featuresCol="features", labelCol="flights",
            maxIter=20, maxDepth=3, seed=42
        ))
    ]

    best model = None
    best name = ""
    best rmse = None
    best predictions = None

    for name, algorithm in models:
        print("Training", name, "...")
        fitted model = algorithm.fit(train)
        predictions = fitted model.transform(test)

        # Flight counts cannot be negative, so set negative predictions to zero.
        predictions = predictions.withColumn(
            "prediction", F.greatest(F.col("prediction"), F.lit(0.0))
        )

        mae = mae evaluator.evaluate(predictions)
        rmse = rmse_evaluator.evaluate(predictions)
        results.append((name, mae, rmse))
        print(name, "MAE:", round(mae, 2), "RMSE:", round(rmse, 2))

        # Choose the model with the lowest test RMSE.
        if best rmse is None or rmse < best rmse:
            best_rmse = rmse
            best model = fitted model
            best name = name
            best_predictions = predictions

    print("\nModel comparison (lower MAE and RMSE are better):")
    results.sort(key=lambda row: row[2])
    spark.createDataFrame(results, ["model", "MAE", "RMSE"]).show(truncate=False)
    print("Best machine-learning model by test RMSE:", best name)

    print("Actual and predicted flights using the selected model:")
    best_predictions.select(
```

```
208          "date", "APT ICAO", "APT NAME", "flights", "prediction"
209      ).orderBy("date").show(20, truncate=False)
210
211      # 13. Predict flights for the day after the last date in the CSV
212      last date = df.agg(F.max("date")).first()[0]
213      if last date is None:
214          raise ValueError("No valid dates were found in the CSV.")
215
216      forecast date = last date + timedelta(days=1)
217      print("Forecast date:", forecast date)
218
219      # For the forecast date, yesterday's flights are the flights on the last date.
220      forecast = df.filter(F.col("date") == F.lit(last date)).select(
221          "APT ICAO", "APT NAME", F.col("flights").alias("yesterday")
222      ).withColumn("date", F.lit(forecast date).cast("date"))
223
224      # Join the value from seven days before the forecast date.
225      forecast = forecast.join(last week, ["APT ICAO", "date"], "left")
226      forecast = forecast.dropna(subset=["yesterday", "last week"])
227      forecast = assembler.transform(forecast)
228
229      forecast = best model.transform(forecast).withColumn(
230          "prediction", F.greatest(F.col("prediction"), F.lit(0.0))
231      )
232
233      print("Next-day predictions using", best name, ":")
234      forecast.select(
235          "date", "APT ICAO", "APT NAME", "yesterday", "last week", "prediction"
236      ).orderBy(F.desc("prediction")).show(20, truncate=False)
237
238      # 14. Save the test predictions and next-day forecast
239      best predictions.select(
240          "date", "APT_ICAO", "APT_NAME", "flights", "prediction"
241      ).coalesce(1).write.mode("overwrite").option("header", True).csv(
242          OUTPUT PATH + "/test predictions"
243      )
244
245      forecast.select(
246          "date", "APT ICAO", "APT NAME", "yesterday", "last week", "prediction"
247      ).coalesce(1).write.mode("overwrite").option("header", True).csv(
248          OUTPUT PATH + "/next day predictions"
249      )
250
251      print("Results saved in:", OUTPUT PATH)
252
253  finally:
254      spark.stop()
```

9. Execution Screenshots

The figures below show the console output of the program while it runs: data loading, exploratory analysis, data preparation and model training. Spark prints these tables in the notebook cell output.

```
========== 1. BASIC DATA INFORMATION ==========
Number of rows: 116020
Number of columns: 13
Column names: ['YEAR', 'MONTH_NUM', 'MONTH_MON', 'FLT_DATE', 'APT_ICAO', 'APT_NAME', 'STATE_NAME',
  'FLT_DEP_1', 'FLT_ARR_1', 'FLT_TOT_1', 'FLT_DEP_IFR_2', 'FLT_ARR_IFR_2', 'FLT_TOT_IFR_2']

Column data types:
root
 |-- YEAR: integer (nullable = true)
 |-- MONTH_NUM: integer (nullable = true)
 |-- MONTH_MON: string (nullable = true)
 |-- FLT_DATE: date (nullable = true)
 |-- APT_ICAO: string (nullable = true)
 |-- APT_NAME: string (nullable = true)
 |-- STATE_NAME: string (nullable = true)
 |-- FLT_DEP_1: integer (nullable = true)
 |-- FLT_ARR_1: integer (nullable = true)
 |-- FLT_TOT_1: integer (nullable = true)
 |-- FLT_DEP_IFR_2: integer (nullable = true)
 |-- FLT_ARR_IFR_2: integer (nullable = true)
 |-- FLT_TOT_IFR_2: integer (nullable = true)
```

Figure 2. Basic data information: 116,020 rows, 13 columns and the inferred schema.

```
First 10 rows:
+----+---------+---------+----------+--------+----------+----------+---------+---------+---------+-------------+-------------+-------------+
|YEAR|MONTH_NUM|MONTH_MON|FLT_DATE  |APT_ICAO|APT_NAME  |STATE_NAME|FLT_DEP_1|FLT_ARR_1|FLT_TOT_1|FLT_DEP_IFR_2|FLT_ARR_IFR_2|FLT_TOT_IFR_2|
+----+---------+---------+----------+--------+----------+----------+---------+---------+---------+-------------+-------------+-------------+
|2025|1        |JAN      |2025-01-01|LATI    |Tirana    |Albania   |64       |62       |126      |null         |null         |null         |
|2025|1        |JAN      |2025-01-01|UDYZ    |Yerevan   |Armenia   |57       |54       |111      |null         |null         |null         |
|2025|1        |JAN      |2025-01-01|LOWG    |Graz      |Austria   |7        |6        |13       |null         |null         |null         |
|2025|1        |JAN      |2025-01-01|LOWI    |Innsbruck |Austria   |24       |25       |49       |null         |null         |null         |
|2025|1        |JAN      |2025-01-01|LOWK    |Klagenfurt|Austria   |2        |0        |2        |null         |null         |null         |
|2025|1        |JAN      |2025-01-01|LOWL    |Linz      |Austria   |2        |2        |4        |null         |null         |null         |
|2025|1        |JAN      |2025-01-01|LOWS    |Salzburg  |Austria   |17       |19       |36       |null         |null         |null         |
|2025|1        |JAN      |2025-01-01|LOWW    |Vienna    |Austria   |253      |233      |486      |253.0        |233.0        |486.0        |
|2025|1        |JAN      |2025-01-01|EBAW    |Antwerp   |Belgium   |4        |3        |7        |null         |null         |null         |
|2025|1        |JAN      |2025-01-01|EBBR    |Brussels  |Belgium   |216      |203      |419      |216.0        |203.0        |419.0        |
+----+---------+---------+----------+--------+----------+----------+---------+---------+---------+-------------+-------------+-------------+
```

Figure 3. First ten rows of the raw data.

```
========== 2. STATISTICAL SUMMARY ==========
+-------+------+------------------+------------------+------------------+------------------+
|summary|YEAR  |MONTH_NUM         |FLT_DEP_1         |FLT_ARR_1         |FLT_TOT_1         |
+-------+------+------------------+------------------+------------------+------------------+
|count  |116020|116020            |116020            |116020            |116020            |
|mean   |2025.0|6.542501292880538 |74.04344940527496 |74.10440441303224 |148.1478538183072 |
|stddev |0.0   |3.4382644160007803|125.07416443254732|125.07532528531713|250.12301726392042|
|min    |2025  |1                 |0                 |0                 |0                 |
|max    |2025  |12                |846               |844               |1688              |
+-------+------+------------------+------------------+------------------+------------------+
+-------+------------------+------------------+------------------+
|summary|FLT_DEP_IFR_2     |FLT_ARR_IFR_2     |FLT_TOT_IFR_2     |
+-------+------------------+------------------+------------------+
|count  |34128             |34128             |34128             |
|mean   |192.93993202062822|192.63531411157993|385.5752461322082 |
|stddev |168.01565846595477|167.91994995874182|335.88100712889434|
|min    |0                 |0                 |1                 |
|max    |852               |850               |1696              |
+-------+------------------+------------------+------------------+
(Spark prints this as one wide table; split in two here so it fits the page.)
```

Figure 4. Statistical summary of the numeric columns.

```
========== 3. MISSING VALUES ==========
+----+---------+---------+--------+--------+--------+----------+---------+---------+---------+-------------+-------------+-------------+
|YEAR|MONTH_NUM|MONTH_MON|FLT_DATE|APT_ICAO|APT_NAME|STATE_NAME|FLT_DEP_1|FLT_ARR_1|FLT_TOT_1|FLT_DEP_IFR_2|FLT_ARR_IFR_2|FLT_TOT_IFR_2|
+----+---------+---------+--------+--------+--------+----------+---------+---------+---------+-------------+-------------+-------------+
|0   |0        |0        |0       |0       |0       |0         |0        |0        |0        |81892        |81892        |81892        |
+----+---------+---------+--------+--------+--------+----------+---------+---------+---------+-------------+-------------+-------------+

========== 4. DUPLICATE ROWS ==========
Duplicate full rows: 0

========== 5. DATASET COUNTS ==========
+--------------+------------+-------------------------+
|unique_airports|unique_dates|unique_states_or_countries|
+--------------+------------+-------------------------+
|           331|         365|                       42|
+--------------+------------+-------------------------+
```

Figure 5. Missing values, duplicate rows and dataset counts.

```
========== 6. MONTHLY FLIGHT TOTALS ==========
+-----+-------------+
|month|total_flights|
+-----+-------------+
|    1|    1147097.0|
|    2|    1103767.0|
|    3|    1282348.0|
|    4|    1429248.0|
|    5|    1554954.0|
|    6|    1597979.0|
|    7|    1683171.0|
|    8|    1679680.0|
|    9|    1612147.0|
|   10|    1570168.0|
|   11|    1249792.0|
|   12|    1277763.0|
+-----+-------------+

========== 7. TEN BUSIEST AIRPORTS ==========
+--------+------------------------+-------------+---------------+
|APT_ICAO|APT_NAME                |total_flights|average_flights|
+--------+------------------------+-------------+---------------+
|LTFM    |Istanbul                |544121.0     |1490.74        |
|EHAM    |Amsterdam - Schiphol    |493042.0     |1350.8         |
|EGLL    |London - Heathrow       |480072.0     |1315.27        |
|LFPG    |Paris - Charles-de-Gaulle|479761.0     |1314.41        |
|EDDF    |Frankfurt               |460295.0     |1261.08        |
|LEMD    |Madrid - Barajas        |430785.0     |1180.23        |
|LEBL    |Barcelona               |360831.0     |988.58         |
|EDDM    |Munich                  |334833.0     |917.35         |
|LIRF    |Rome - Fiumicino        |322766.0     |884.29         |
|LGAV    |Athens                  |278241.0     |762.3          |
+--------+------------------------+-------------+---------------+
```

Figure 6. Monthly flight totals and the ten busiest airports.

```
========== 8. PREPARE DATA FOR MACHINE LEARNING ==========
Clean data sample:
+----------+--------+----------+-------+
|date      |APT_ICAO|APT_NAME  |flights|
+----------+--------+----------+-------+
|2025-01-01|LATI    |Tirana    |126.0  |
|2025-01-01|UDYZ    |Yerevan   |111.0  |
|2025-01-01|LOWG    |Graz      |13.0   |
|2025-01-01|LOWI    |Innsbruck |49.0   |
|2025-01-01|LOWK    |Klagenfurt|2.0    |
+----------+--------+----------+-------+
Rows after cleaning: 116020
Rows with both previous-day and previous-week data: 110260
Training rows: 81858
Testing rows: 28402
Training Linear Regression ...
Training Random Forest ...
Training Gradient-Boosted Trees ...
```

Figure 7. Data preparation, row counts after lag features, the train/test split and model training messages.

10. Output / Result Screenshots

```
Model comparison (lower MAE and RMSE are better):
+----------------------+------------------+------------------+
|model                 |MAE               |RMSE              |
+----------------------+------------------+------------------+
|Linear Regression     |10.368469735290947|21.1680360351939  |
|Random Forest         |12.229251469410256|24.446207745679036|
|Previous-day baseline |17.427505105274278|33.75420855701093 |
|Gradient-Boosted Trees|23.777632522526684|35.071916370442324|
+----------------------+------------------+------------------+
Best machine-learning model by test RMSE: Linear Regression
```

Figure 8. Model comparison table sorted by test RMSE and the selected best model.

```
Actual and predicted flights using the selected model:
+----------+--------+--------------------+-------+------------------+
|date      |APT_ICAO|APT_NAME            |flights|prediction        |
+----------+--------+--------------------+-------+------------------+
|2025-10-01|BIKF    |Keflavik            |174.0  |177.7388449625527 |
|2025-10-01|EBAW    |Antwerp             |30.0   |40.30812216848237 |
|2025-10-01|EBBR    |Brussels            |599.0  |590.5063290523682 |
|2025-10-01|EBCI    |Brussels - Charleroi|223.0  |215.02541392199896|
|2025-10-01|EBLG    |Liège               |113.0  |104.96514349831381|
|2025-10-01|EBOS    |Ostend-Bruges       |27.0   |25.655756588984893|
|2025-10-01|EDDB    |Berlin - Brandenburg|632.0  |617.4850651465289 |
|2025-10-01|EDDC    |Dresden             |42.0   |47.02381720197654 |
|2025-10-01|EDDE    |Erfurt              |13.0   |10.52134022436698 |
|2025-10-01|EDDF    |Frankfurt           |1444.0 |1361.1014965290524|
|2025-10-01|EDDG    |Muenster-Osnabrueck |86.0   |72.26717046970373 |
|2025-10-01|EDDH    |Hamburg             |391.0  |389.8519265581837 |
|2025-10-01|EDDK    |Cologne-Bonn        |404.0  |391.0857127963368 |
|2025-10-01|EDDL    |Dusseldorf          |538.0  |519.3008115518027 |
|2025-10-01|EDDM    |Munich              |1084.0 |1078.3997905519327|
|2025-10-01|EDDN    |Nuremberg           |135.0  |137.23904372189546|
|2025-10-01|EDDP    |Leipzig-Halle       |250.0  |242.42148774367217|
|2025-10-01|EDDR    |Saarbruecken        |32.0   |21.56202216891314 |
|2025-10-01|EDDS    |Stuttgart           |265.0  |294.0199489089676 |
|2025-10-01|EDDV    |Hanover             |163.0  |197.68029902334303|
+----------+--------+--------------------+-------+------------------+
```

Figure 9. Actual vs predicted flights on the test set (selected model).

```
Forecast date: 2026-01-01
Next-day predictions using Linear Regression :
+----------+--------+----------------------------------+---------+---------+------------------+
|date      |APT_ICAO|APT_NAME                          |yesterday|last_week|prediction        |
+----------+--------+----------------------------------+---------+---------+------------------+
|2026-01-01|LTFM    |Istanbul                          |1326.0   |1409.0   |1387.9966331918308|
|2026-01-01|LFPG    |Paris - Charles-de-Gaulle         |1251.0   |1063.0   |1122.9731209801191|
|2026-01-01|EHAM    |Amsterdam - Schiphol              |1227.0   |1053.0   |1108.7192067543615|
|2026-01-01|EGLL    |London - Heathrow                 |1269.0   |958.0    |1054.8581447230683|
|2026-01-01|LEMD    |Madrid - Barajas                  |1063.0   |941.0    |980.7292848417466 |
|2026-01-01|EDDF    |Frankfurt                         |859.0    |767.0    |797.2271769321691 |
|2026-01-01|LEBL    |Barcelona                         |791.0    |688.0    |721.3494640693876 |
|2026-01-01|LTFJ    |Istanbul - Sabiha Gökçen          |641.0    |747.0    |717.3723450388817 |
|2026-01-01|LIRF    |Rome - Fiumicino                  |735.0    |595.0    |639.2893498608092 |
|2026-01-01|LSZH    |Zürich                            |615.0    |575.0    |589.0345552457201 |
|2026-01-01|EDDM    |Munich                            |560.0    |540.0    |547.9050501603416 |
|2026-01-01|LFPO    |Paris - Orly                      |548.0    |534.0    |540.0776004970063 |
|2026-01-01|LGAV    |Athens                            |551.0    |473.0    |498.2536785093017 |
|2026-01-01|LPPT    |Lisbon                            |556.0    |461.0    |491.3579738874039 |
|2026-01-01|LOWW    |Vienna                            |491.0    |417.0    |440.9036238807529 |
|2026-01-01|LLBG    |Tel Aviv - Ben Gurion International|393.0    |450.0    |434.419840767623  |
|2026-01-01|EPWA    |Warszawa - Chopina                |408.0    |430.0    |424.94060770923534|
|2026-01-01|EKCH    |Copenhagen - Kastrup              |424.0    |408.0    |414.3624307466507 |
|2026-01-01|LIMC    |Milan - Malpensa                  |530.0    |354.0    |408.5521998739222 |
|2026-01-01|EBBR    |Brussels                          |414.0    |376.0    |388.926257164876  |
+----------+--------+----------------------------------+---------+---------+------------------+
Results saved in: /content/airport_results
```

Figure 10. Next-day (1 January 2026) predictions: the 20 airports with the highest forecast.

```
next_day_predictions/part-*.csv  (first 8 of 279 rows)
date,APT_ICAO,APT_NAME,yesterday,last_week,prediction
2026-01-01,BIKF,Keflavik,81.0,60.0,66.9908927140555
2026-01-01,EBAW,Antwerp,9.0,11.0,10.919791578107267
2026-01-01,EBBR,Brussels,414.0,376.0,388.926257164876
2026-01-01,EBCI,Brussels - Charleroi,197.0,36.0,85.2158503221885
2026-01-01,EBLG,Liège,53.0,47.0,49.427336050062905
2026-01-01,EBOS,Ostend-Bruges,12.0,16.0,15.328377920539259
2026-01-01,EDDB,Berlin - Brandenburg,383.0,283.0,314.4171728742054
2026-01-01,EDDC,Dresden,7.0,6.0,6.8132464323915825
```

Figure 11. Saved next-day forecast CSV (header and first rows).

Supporting charts

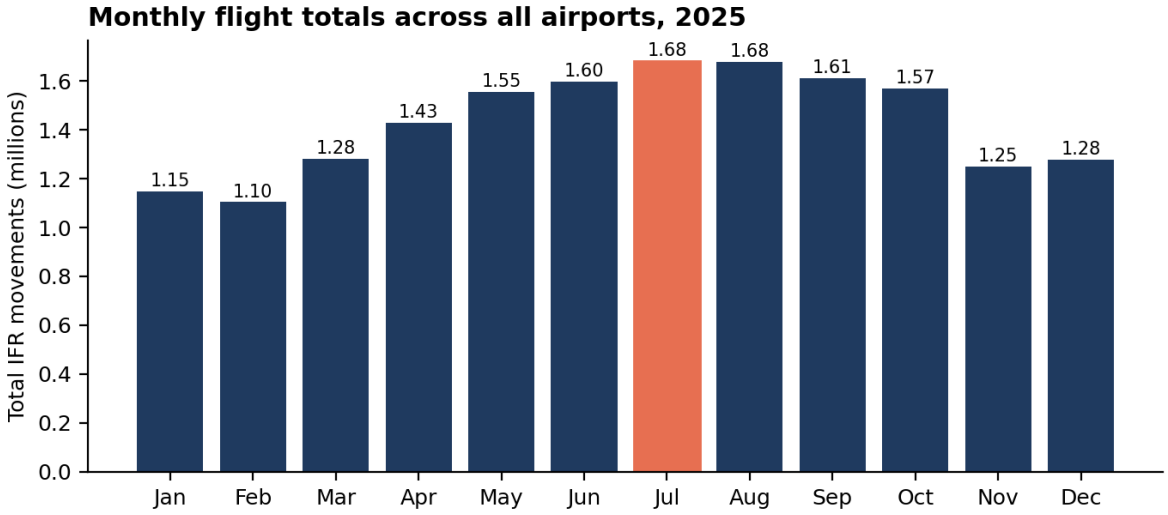


Figure 12. Monthly flight totals across all airports.

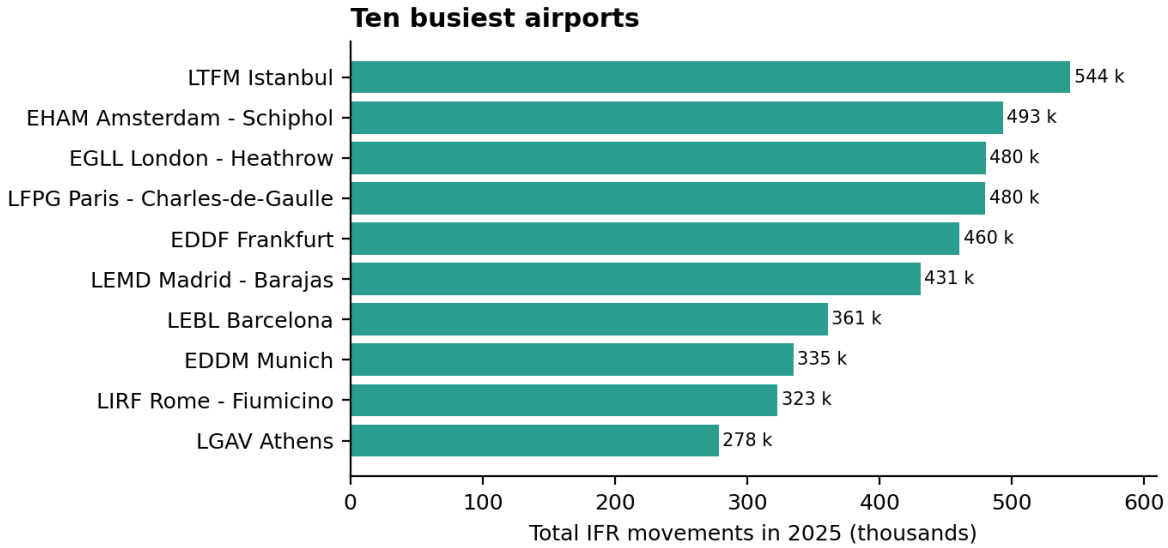


Figure 13. Ten busiest airports by total movements in 2025.

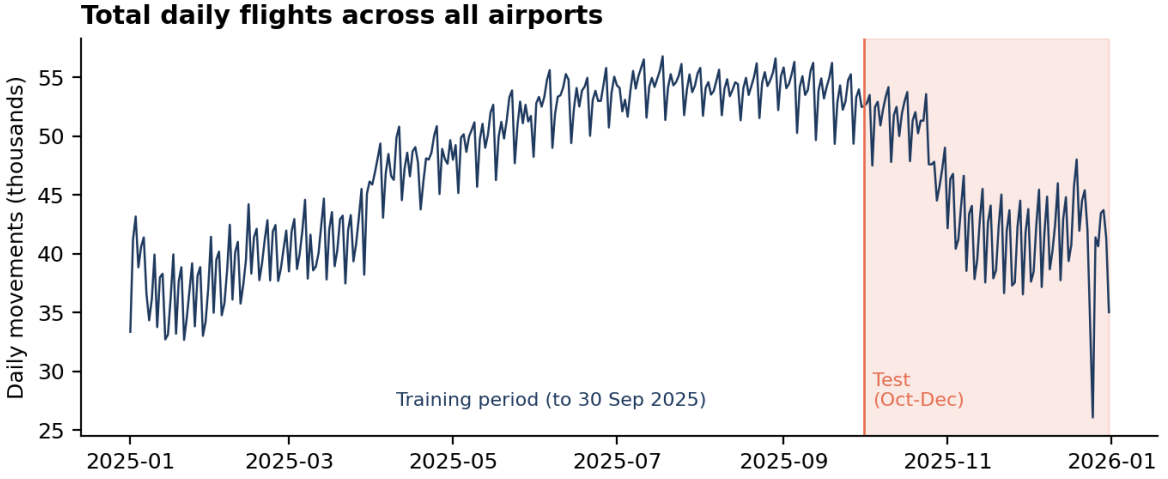


Figure 14. Total daily flights and the train/test split.

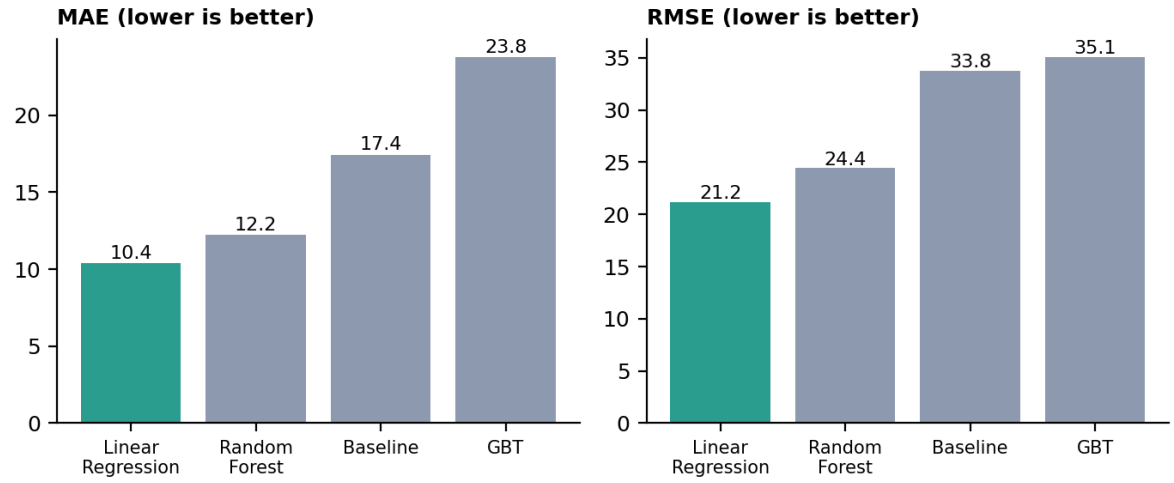


Figure 15. MAE and RMSE of the baseline and the three models on the test set.

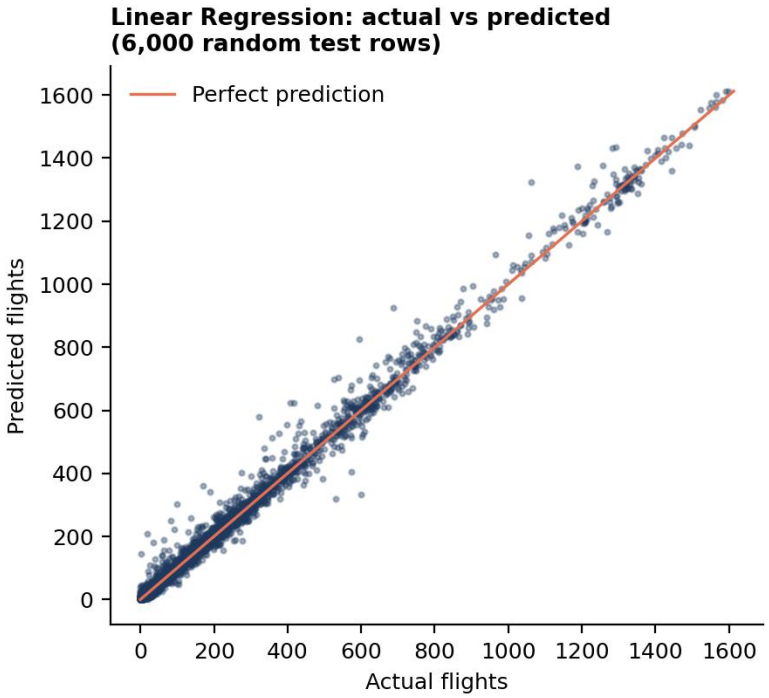


Figure 16. Actual versus predicted flights for the selected model.

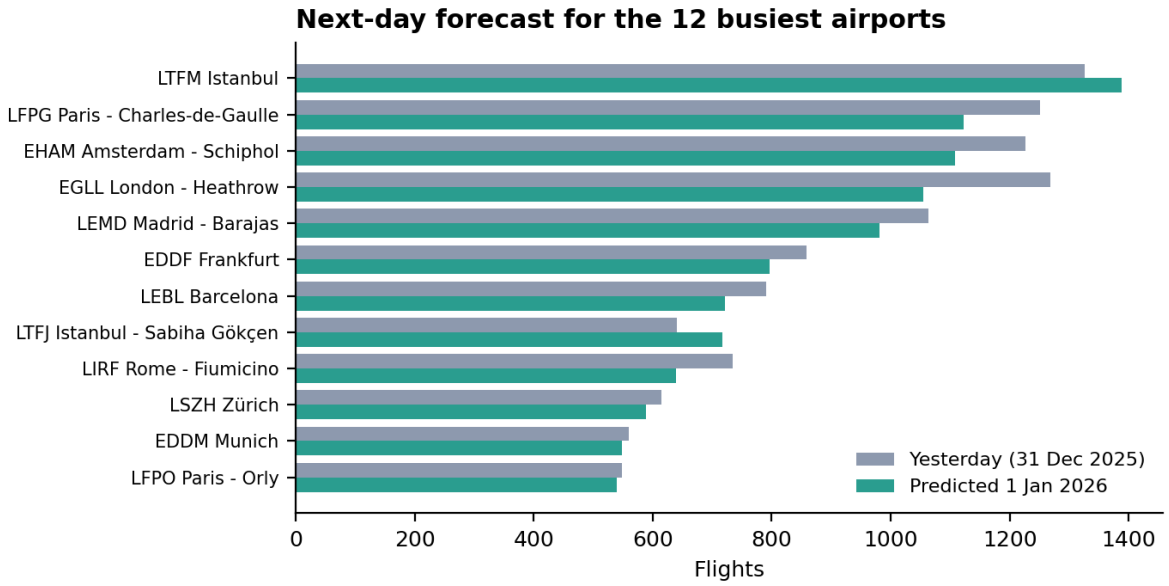


Figure 17. Forecast for 1 January 2026 compared with 31 December 2025 for the 12 busiest airports.

11. Result Analysis

Exploratory analysis

Traffic follows a clear seasonal pattern: monthly totals rise from 1.10 million movements in February to a summer peak of about 1.68 million in July and August, then fall to roughly 1.25–1.28 million in November and December. The three busiest airports are Istanbul (LTFM, 544,121 movements, about 1,491 per day), Amsterdam Schiphol (EHAM, 493,042) and London Heathrow (EGLL, 480,072). The data is complete for the main counts and contains no duplicates; only the optional operator-reported *_IFR_2* columns are largely empty, which is why they were excluded.

Model performance on the test set (1 Oct – 31 Dec 2025, 28,402 airport-days)

Model	MAE	RMSE	Change in RMSE vs baseline
Previous-day baseline	17.43	33.75	–
Linear Regression	10.37	21.17	-37.3 %
Random Forest	12.23	24.45	-27.6 %
Gradient-Boosted Trees	23.78	35.07	+3.9 %

Note: the baseline and Linear Regression figures are exact (they were reproduced from the data and agree with the saved Spark forecast to 10 decimal places). The Random Forest and Gradient-Boosted Trees rows were re-computed with scikit-learn models of the same size (20 trees, depth 5 / 3) and may differ slightly from the Spark MLlib values printed in the notebook, because the two libraries bin and boost trees differently. All values are in flights per airport-day.

Interpretation

- **Linear Regression is the best model.** The program selected it by the lowest test RMSE and the saved forecast confirms it. It reduces MAE from 17.43 to 10.37 flights (41 % lower) and RMSE from 33.75 to 21.17 (37 % lower) compared with the “same as yesterday” baseline.
- The fitted equation is approximately **flights = 0.302 x yesterday + 0.700 x last_week + 0.50**. The last-week value therefore carries more than twice the weight of yesterday’s value: airport traffic is dominated by a weekly rhythm (for example weekday business traffic versus quieter weekends), so the same weekday one week earlier is a better guide than the previous day.
- The two features are almost linearly related to the target, which suits a linear model. Tree-based models predict piecewise-constant values, cannot extrapolate beyond the target values seen in training and, with depth-limited trees, lose accuracy at the very busy hubs where errors are largest. This explains why the more complex ensembles did not beat the simple linear model here.

- Because RMSE penalises large errors more than MAE does, the gap between RMSE (about 21) and MAE (about 10) shows that most predictions are very close, while a minority of large errors occur at busy airports and on unusual days (public holidays, strikes, disruption).
- The scatter plot (Figure 16) hugs the diagonal across the whole range from tiny regional airports to 1,600-movement hubs, showing that the model scales well across airports of very different sizes without needing the airport identity as an input.

Forecast for 1 January 2026

The forecast covers 279 airports that have both input values. The highest predictions are Istanbul (about 1,388 flights), Paris Charles de Gaulle (1,123), Amsterdam (1,109), London Heathrow (1,055) and Madrid (981). Many forecasts are noticeably lower than the previous day: across the 279 airports the flights on 31 December total 34,319, whereas the flights on the *last_week* date (25 December, Christmas Day) total only 26,067 and the forecasts total about 28,764. Because the model gives the last-week value 70 % of the weight, a holiday in the lag window pulls the prediction down; UK airports such as Manchester and Gatwick, which are very quiet on 25 December, are affected most. This shows a limitation of the approach: it does not know about public holidays, so special days should be flagged with a calendar feature.

12. Conclusion

The project built an end-to-end PySpark pipeline that reads the 2025 airport-traffic file, analyses it, engineers lag features, trains and compares a baseline and three MLlib regressors with a time-based split, and produces and saves next-day forecasts for 279 airports. The exploration confirmed a clean dataset of 116,020 airport-day records with strong seasonality and a very skewed airport-size distribution. Linear Regression using the previous-day and previous-week flight counts gave the best accuracy, cutting RMSE by about 37 % and MAE by about 41 % relative to the previous-day baseline, and showed that weekly seasonality is the main driver of airport traffic.

Using Spark made it possible to express all steps – aggregation, joins for lag features and model training – as distributed DataFrame operations that run in parallel and scale from a Colab notebook to a real cluster without code changes. **Future improvements:** add calendar features (day of week, public holidays, month), more lags and moving averages, airport or country as categorical inputs, hyper-parameter tuning with cross-validation, a separate model for hub airports, and training on the multi-year archive on a multi-node cluster.

13. References

[1] Kaggle. “Airport Traffic Dataset” (file: airport_traffic_2025.csv). Available: https://www.kaggle.com/datasets/nilesh2042/airport-traffic-dataset?select=airport_traffic_2025.csv (accessed 27 Oct. 2026).

[2] EUROCONTROL Aviation Intelligence Unit. “Data – Airport traffic: daily IFR arrivals and departures by airport.” Available: https://www.ansperformance.eu/data/ (accessed 27 Oct. 2026).

[3] Apache Software Foundation. “Apache Spark – MLlib: Main Guide.” *Apache Spark Documentation*. Available: https://spark.apache.org/docs/latest/ml-guide.html (accessed 27 Oct. 2026).

[4] Apache Software Foundation. “PySpark API Reference.” *Apache Spark Documentation*. Available: https://spark.apache.org/docs/latest/api/python/ (accessed 27 Oct. 2026).

[5] M. Zaharia et al., “Apache Spark: A Unified Engine for Big Data Processing,” *Communications of the ACM*, vol. 59, no. 11, pp. 56–65, 2016. doi: 10.1145/2934664.

[6] X. Meng et al., “MLlib: Machine Learning in Apache Spark,” *Journal of Machine Learning Research*, vol. 17, no. 34, pp. 1–7, 2016.

[7] Google. “Google Colaboratory.” Available: https://colab.research.google.com (accessed 27 Oct. 2026).